SecurePass: A Minimalist RESTful Password Service with C Integration

Author: Student **Course:** Software Development / IT Security **Institution:** Hochschule Offenburg (HSO) **Repository:** https://edugit.hs-offenburg.de **Submission:** Lab Task 1 Report

1. Introduction

Passwords are the primary barrier defending user accounts against unauthorized access. Creating secure passwords requires genuine cryptographic randomness, while storing passwords securely requires slow, salted hashing to prevent offline brute-force attacks.

The objective of this lab task is to design and implement **SecurePass**, a minimalist password service that satisfies all core requirements specified in the assignment: 1. **Configurable Password Generation:** Generates passwords based on parameters (length, uppercase, lowercase, numbers, symbols) using the cryptographic `secrets` module. 2. **REST Web Service via Flask:** Implements a clean REST API running on a lightweight Flask web server. 3. **Data Persistence via SQLite:** Stores and manages saved passwords in a local SQLite database (`passwords.db`). 4. **PBKDF2 Password Hashing:** Uses Python's standard `secrets` (for random salts) and `hashlib` (for PBKDF2-HMAC-SHA256 hashing) with constant-time verification. 5. **C Programming Integration:** Calls a native C function (`password_check.c`) from Python using `ctypes` to compute a password strength score.

2. System Architecture & REST Service Design

The project is structured with a minimalist, single-tier organization where every component has a distinct purpose:

```
[ Web Browser / REST Client ]
            │  HTTP / JSON
            ▼
     [ Flask (app.py) ]
     ├── Generator Function (uses Python 'secrets')
     ├── PBKDF2 Hasher (uses 'secrets' & 'hashlib')
     ├── C ctypes Foreign Function Interface (calls password_check.dll)
     └── SQLite Persistence Layer (database.py)
```

REST API Endpoints

The Flask application (`app.py`) exposes the following endpoints:

Method	Endpoint	Description	Status Code
`GET`	`/`	Serves the single-page HTML/CSS dashboard	`200 OK`
`POST`	`/api/generate`	Generates a random password based on parameters	`200 OK`
`POST`	`/api/hash`	Computes a PBKDF2 hash using salt and hashlib	`200 OK`
`POST`	`/api/verify`	Verifies a candidate password in constant-time	`200 OK`
`POST`	`/api/check_strength`	Directly invokes the C function via ctypes	`200 OK`
`GET`	`/api/passwords`	Retrieves all saved passwords from SQLite	`200 OK`
`POST`	`/api/passwords`	Persists a new password entry to SQLite	`201 Created`
`DELETE`	`/api/passwords/`	Deletes a stored password from SQLite	`200 OK`

3. Password Generation with CSPRNG (`secrets`)

In Python, the standard `random` module uses the Mersenne Twister algorithm. While fast for simulations, it is deterministic and predictable; if an attacker observes a sequence of outputs, they can reconstruct internal generator state and predict subsequent passwords.

To ensure true unpredictability, SecurePass uses the Python standard library **`secrets`** module, which accesses operating system entropy (`CryptGenRandom` on Windows or `/dev/urandom` on Linux): - **`secrets.choice(pool)`:** Picks characters from

active pools (uppercase, lowercase, digits, symbols) with cryptographically secure random distribution. - **Guarantee Policy:** Ensures at least one character from every active set is selected first. - **`secrets.SystemRandom().shuffle()`:** Cryptographically shuffles the final character list so that the guaranteed characters are randomly placed.

---

4. PBKDF2 Hashing using `secrets` and `hashlib`

Storing passwords in plaintext or with simple fast hashes (such as MD5 or single-round SHA-256) is dangerous because modern GPUs can compute billions of hashes per second. SecurePass implements **PBKDF2** (Password-Based Key Derivation Function 2) according to security best practices.

4.1 Salt Generation (`secrets`)

A salt is a unique, random string generated for every password. It ensures that identical passwords yield completely different hashes, neutralizing pre-computed rainbow table attacks:

```
salt_hex = secrets.token_hex(16)
```

4.2 Key Derivation (`hashlib.pbkdf2_hmac`)

The hash is computed using `hashlib.pbkdf2_hmac`:

```
derived_bytes = hashlib.pbkdf2_hmac(
    hash_name="sha256",
    password=password.encode("utf-8"),
    salt=bytes.fromhex(salt_hex),
    iterations=100000
)
```

The 100,000 iterations intentionally introduce a computational work factor, making brute-force dictionary attacks impractical.

4.3 Constant-Time Verification

Standard string comparison (`==`) exits immediately at the first non-matching byte, which leaks timing information (side-channel attack). SecurePass verifies passwords in constant time using:

```
secrets.compare_digest(candidate_hex, expected_hash_hex)
```

---

5. Calling a Native C Function from Python (`ctypes`)

To revise C programming and demonstrate cross-language interoperability, a native C function was developed in `c_module/password_check.c`.

5.1 What the C Function Does

The C function `check_password_strength()` analyzes a password string directly in memory: 1. It iterates through the string using a fast C pointer loop. 2. It detects character categories using standard C library functions (`isupper`, `islower`, `isdigit`). 3. It calculates a score from 0 to 100 based on length (4 points per character) and variety bonuses (10 for uppercase, 10 for lowercase, 10 for digits, 15 for special symbols).

```
EXPORT int check_password_strength(const char *password) {
    if (!password) return 0;
    int len = (int)strlen(password);
    int has_upper = 0, has_lower = 0, has_digit = 0, has_special = 0;
    for (int i = 0; i < len; i++) {
        unsigned char c = (unsigned char)password[i];
        if (isupper(c)) has_upper = 1;
        else if (islower(c)) has_lower = 1;
        else if (isdigit(c)) has_digit = 1;
        else has_special = 1;
    }
    int score = len * 4;
    if (has_upper) score += 10;
    if (has_lower) score += 10;
    if (has_digit) score += 10;
    if (has_special) score += 15;
    return (score > 100) ? 100 : score;
}
```

5.2 Python Binding via `ctypes`

Python binds to the compiled shared library (`password_check.dll` on Windows / `password_check.so` on Linux) using Python's built-in `ctypes` module:

```
import ctypes
c_lib = ctypes.CDLL("./c_module/password_check.dll")
c_lib.check_password_strength.argtypes = [ctypes.c_char_p]
c_lib.check_password_strength.restype = ctypes.c_int

# Invoking the C function from Python
score = c_lib.check_password_strength(password.encode("utf-8"))
```

Python passes the string as a UTF-8 encoded byte array (`const char*`), and receives the integer score directly from compiled C machine code.

---

6. SQLite Database Persistence

Data persistence is implemented in `database.py` using SQLite3 (`passwords.db`).

Database Schema

```
CREATE TABLE IF NOT EXISTS passwords (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    service_name TEXT NOT NULL,
    username TEXT NOT NULL,
    password_value TEXT NOT NULL,
    hash_value TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Protection Against SQL Injection

All queries utilize parameterized SQL syntax with `?` place-holders:

```
cursor.execute("INSERT INTO passwords (service_name, username, password_value, hash_value) VALU
               (service_name, username, password, hash_val))
```

This guarantees that user input is never concatenated directly into SQL strings, eliminating SQL injection vulnerabilities.

---

7. Automated Testing & Verification

A comprehensive test suite was implemented in `test_app.py` using Python's standard `unittest` framework: - **`test_generator`:** Verifies length configuration and random output validity. - **`test_pbkdf2_hashing_and_verify`:**

Validates salt generation, key derivation, and matching vs non-matching verification. - **`test_c_function`:** Confirms the C library loads successfully and computes correct strength scores. - **`test_sqlite_persistence`:** Tests saving, listing, and deleting records in SQLite. - **`test_rest_api_*`:** Verifies HTTP endpoints (`/api/generate`, `/api/hash`, `/api/verify`, `/api/check_strength`).

Execution Result:

```
Ran 7 tests in 0.238s - OK (100% Passing)
```

---

8. Conclusion

The SecurePass project successfully implements a clean, robust, and minimal password service that fulfills every requirement of Lab Task 1: - Secure generation via `secrets` - PBKDF2 hashing via `secrets` and `hashlib` - Data persistence via SQLite - Native C function integration via `ctypes` - REST service hosted on Flask